

Name : KEERTHANA R

Reg No : 411625104031

Department : CSE

Subject : JAVA (OOPS)

Batch : 4

Title : Chocolate Program Demonstrating Compile-time and Run-time Polymorphism

Aim

To design and implement a Java program that demonstrates both compile-time polymorphism (method overloading) and run-time polymorphism (method overriding) using a chocolate example.

Procedure

Step 1: Creating Base Class

- A class named Chocolate was created.
- It contains overloaded methods showDetails() to demonstrate compile-time polymorphism.
- It also has a displayInfo() method to be overridden by subclasses.

Step 2: Implementing Subclasses

- A subclass Brownie extends Chocolate and overrides displayInfo() to show brownie-specific details.
- Another subclass IceCream extends Chocolate and overrides displayInfo() to show ice-cream-specific details.

Step 3: Designing the Main Class

- The Main class contains the main() method.
- Objects of Brownie and IceCream are created but referenced using the parent class Chocolate.

- This demonstrates run-time polymorphism.
- The overloaded showDetails() methods are invoked to demonstrate compile-time polymorphism.

Step 4: Applying Object-Oriented Concepts

- **Inheritance** – Brownie and IceCream extend Chocolate.
- **Polymorphism** –
 - Compile-time: Method overloading (showDetails).
 - Run-time: Method overriding (displayInfo).
- **Encapsulation** – Attributes like ingredients and pieces are maintained inside their respective classes.

Step 5: Executing the Program

- The program creates chocolate objects and calls both overridden and overloaded methods.
- Output is displayed based on the type of chocolate and the method invoked.

Step 6: Testing

- Tested with different chocolate types (Brownie, IceCream).
- Verified correct outputs for overridden methods.
- Verified compile-time polymorphism by calling overloaded methods with different argument types.

Code

```
import java.util.*;
```

```
class Chocolate {
```

```
String name;
String country;
Chocolate(String name, String country) {
    this.name = name;
    this.country = country;
}
void showDetails(int price) {
    System.out.println(name + " costs " + price + " rupees.");
}
void showDetails(String combo) {
    System.out.println(name + " is " + combo);
}
void showDetails(int ingredients, int pieces) {
    System.out.println(name + " contains " + ingredients + " ingredients and has " + pieces + " pieces.");
}
void displayInfo() {
    System.out.println("Chocolate name: " + name + " from " + country);
}
}
```

```
class Brownie extends Chocolate {
    int ingredients;

    Brownie(String name, String country, int ingredients) {
        super(name, country);
        this.ingredients = ingredients;
    }

    void displayInfo() {
        super.displayInfo();
        System.out.println("Number of ingredients added in the chocolate: " + ingredients);
    }
}

class IceCream extends Chocolate {
    int pieces;

    IceCream(String name, String country, int pieces) {
        super(name, country);
        this.pieces = pieces;
    }

    void displayInfo() {
        super.displayInfo();
```

```
        System.out.println("The chocolate ice cream contains: " + pieces + " pieces");
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Chocolate c1 = new Brownie("Dark Chocolate", "India", 4);
        Chocolate c2 = new IceCream("White Chocolate", "US", 3);

        c1.displayInfo();
        c2.displayInfo();

        c1.showDetails(25);
        c1.showDetails("sweeter");
        c2.showDetails(5, 3);

        sc.close();
    }
}
```

The screenshot shows an online Java compiler interface. On the left, a code editor displays a Java program. The program defines a base class `Chocolate` with methods `showDetails(int price)`, `showDetails(String combo)`, `showDetails(int ingredients, int pieces)`, and `displayInfo()`. It also defines two subclasses: `Brownie` (which extends `Chocolate` and overrides `displayInfo()`) and `IceCream` (which extends `Chocolate` and overrides `displayInfo()`). The `Main` class contains a `main` method that creates instances of `Chocolate`, `Brownie`, and `IceCream` and calls their `displayInfo()` methods.

```
1 import java.util.*;
2
3 class Chocolate {
4     String name;
5     String country;
6     Chocolate(String name, String country) {
7         this.name = name;
8         this.country = country;
9     }
10    void showDetails(int price) {
11        System.out.println(name + " costs " + price + " rupees.");
12    }
13    void showDetails(String combo) {
14        System.out.println(name + " is " + combo);
15    }
16    void showDetails(int ingredients, int pieces) {
17        System.out.println(name + " contains " + ingredients + " ingredients and has " + pieces + " pieces.");
18    }
19    void displayInfo() {
20        System.out.println("Chocolate name: " + name + " from " + country);
21    }
22 }
23 class Brownie extends Chocolate {
24     int ingredients;
25     Brownie(String name, String country, int ingredients) {
26         super(name, country);
27         this.ingredients = ingredients;
28     }
29     void displayInfo() {
30         super.displayInfo();
31         System.out.println("Number of ingredients added in the chocolate: " + ingredients);
32     }
33 }
34 class IceCream extends Chocolate {
35     int pieces;
36     IceCream(String name, String country, int pieces) {
37         super(name, country);
38         this.pieces = pieces;
39     }
40     void displayInfo() {
41         super.displayInfo();
42         System.out.println("The chocolate ice cream contains: " + pieces + " pieces");
43     }
44 }
45 public class Main {
46     public static void main(String[] args) {
```

On the right side of the IDE, the 'Output' tab is active, showing the following text:

```
Chocolate name: Dark Chocolate from India
Number of ingredients added in the chocolate: 4
Chocolate name: White Chocolate from US
The chocolate ice cream contains: 3 pieces
Dark Chocolate costs 25 rupees.
Dark Chocolate is sweeter
White Chocolate contains 5 ingredients and has 3 pieces.
|
```

Below the output, a status bar indicates: "Compiled and executed in 1.735 sec(s)".

Result

The program was successfully executed. It clearly demonstrates **compile-time polymorphism** through method overloading and **run-time polymorphism** through method overriding, using a chocolate example.

Would you like me to also prepare a **sample output screenshot (text output)** section, so your lab record looks complete with expected results?

